Building Highly Available Hadoop Cluster

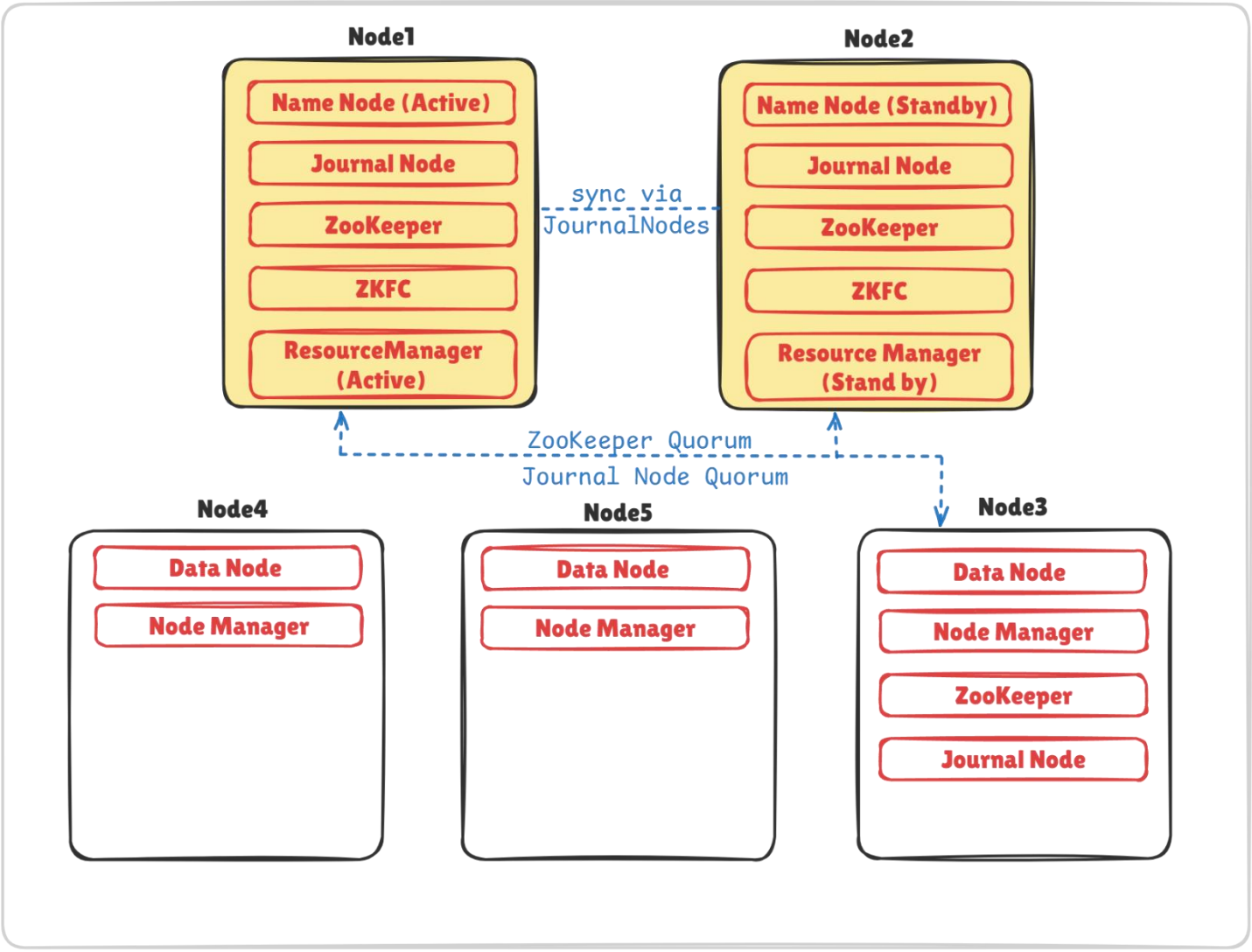
Zeyad Mostafa

—

—

Cluster Architecture

Highly Available Hadoop Cluster



Cluster Architecture

Hadoop HA Cluster — 5 Nodes

HIGH AVAILABILITY · HDFS + YARN · AUTOMATIC FAILOVER

NameNode · ZooKeeper / ZKFC · JournalNode · ResourceManager · DataNode / NodeManager

MASTER NODES

node01 — ACTIVE NAMENODE

- NameNode — NN
- ZK Failover Controller — ZKFC
- ZooKeeper — ZK
- JournalNode — JN
- ResourceManager — RM

node02 — STANDBY NAMENODE

- NameNode — NN
- ZK Failover Controller — ZKFC
- ZooKeeper — ZK
- JournalNode — JN
- ResourceManager — RM

SHARED INFRASTRUCTURE

🔐 ZOOKEEPER QUORUM: node01 — node02 — node03 — 3 nodes · port 2181 · leader election

📔 JOURNALNODE QUORUM: node01 — node02 — node03 — 3 nodes · port 8485 · shared edit log

WORKER NODES

node03 — WORKER

- DataNode — DN
- NodeManager — NM
- ZooKeeper — ZK
- JournalNode — JN

node04 — WORKER

- DataNode — DN
- NodeManager — NM

node05 — WORKER

- DataNode — DN
- NodeManager — NM

⚡ AUTO FAILOVER

node01 Active NN ⇄ node02 Standby NN · ZKFC monitors heartbeat → ZooKeeper elects new leader → JN syncs edits

Architecture Decisions

Why Two Master Nodes (node01 & node02)?

The biggest problem in standard Hadoop is the Name Node is a **single point of failure** so if it crashes, the entire cluster stops. Nobody can read or write data.

The solution is two Name Nodes:

- node01 → Active (doing the actual work)
- node02 → Standby (watching and staying in sync, ready to take over)

If node01 crashes, node02 becomes Active automatically in seconds.

Why ZooKeeper on Exactly 3 Nodes?

ZooKeeper makes decisions by **majority vote**. With 3 nodes, 2 must agree. This means:

- You can lose 1 node and still have majority (2 out of 3)
- The cluster keeps running

Why not 2 nodes? If one crashes, you have 1 out of 2 so there is no majority, ZooKeeper stops. Why not 4 nodes? If 2 crash, you have 2 out of 4 also there is no majority, still stops. Even numbers are dangerous. Why not 5 nodes? You could, but it's more machines for the same result. 3 is the minimum that works.

Why JournalNodes on the Same 3 Nodes as ZooKeeper?

JournalNodes also need a quorum of 3. Instead of adding 3 dedicated machines just for JournalNodes, we put them on the same 3 nodes that already run ZooKeeper.

This saves resources while maintaining both quorums. Same machines, two responsibilities.

Why node01 and node02 Run Zookeeper and Journal Nodes?

These are the most stable nodes in the cluster because they're master nodes that we carefully protect. We never put heavy storage workloads on them. Since they're stable and always running, they're the best candidates to host coordination services.

Why node04 and node05 Are Pure Workers?

These nodes only run Data Node and NodeManager no coordination responsibilities.

- If node04 or node05 crashes → we lose some storage and compute capacity, but the cluster keeps running
- The quorums are unaffected
- No data is permanently lost (with replication factor 3 in production)

They're designed to be **expendable**.

Why ZKFC on node01 and node02 Only?

ZKFC (ZooKeeper Failover Controller) is a watchdog that monitors the NameNode. It makes no sense on worker nodes because there's no NameNode to monitor there.

ZKFC must sit **on the same machine** as the NameNode it monitors it checks health locally and acts immediately. Remote monitoring would be too slow for automatic failover.

Why ResourceManager on node01 and node02?

Same HA logic as NameNode. ResourceManager is the brain of YARN so if it crashes, no new jobs can be submitted. So, we run one Active ResourceManager on node01 and one Standby on node02.

They share the same master machines because:

- These machines are already stable and protected
- No point adding dedicated machines just for ResourceManager
- They have enough RAM and CPU since they don't store data

Why DataNodes on node03, node04, node05 Only?

Master nodes (node01, node02) don't run DataNodes for two reasons:

Performance: NameNode keeps the entire filesystem metadata in RAM. Adding DataNode storage workloads would compete for that RAM and slow down metadata operations.

Safety: If a master node crashes during failover, we don't want to lose data blocks at the same time. Keeping master nodes free of data responsibilities means a master crash only affects coordination, not storage.

Challenges & Solutions

1. NameNode Format Order Error

When trying to initialize the shared edits, got an error saying "NameNode is not formatted." Tried to run `initializeSharedEdits` before formatting the NameNode.

Solution: Must format the NameNode first with `hdfs namenode -format`, then run `hdfs namenode -initializeSharedEdits`. The initialization needs the cluster ID that format creates.

---

2. MapReduce Job Failing with ClassNotFoundException

The word count job failed because NodeManagers couldn't find Hadoop's MapReduce libraries.

Error: Could not find or load main class MRAppMaster.

Solution: Added HADOOP_MAPRED_HOME=/usr/local/hadoop environment variable to mapred-site.xml on all nodes and restarted NodeManagers.

---

3. JournalNode Timing Issue in Automation

When automating startup with scripts, the NameNode on node01 started before node02 and node03's JournalNodes were ready so causing it to crash immediately with "flush failed for required journal."

Solution: Added a wait loop using netcat that checks all 3 JournalNode ports (8485) before starting the NameNode. The script retries every 3 seconds until all 3 are reachable.

---

4. Port Already in Use

After killing a DataNode, port 9866 remained occupied by a ghost process invisible to jps. New DataNode startup failed with "Address already in use."

Solution: Installed net-tools and used netstat to find the process ID holding the port, then killed it as root before restarting the DataNode.

---

5. Stale PID Files Blocking Daemon Restarts

After processes crashed or were killed, they left behind PID files in /tmp. Hadoop read these files and thought the service was still running, refusing to start again with 'already running as process X'.

Solution: Fixed by deleting stale PID files from /tmp and clearing /tmp/hsperfdata_hadoop/ which held cached Java process information causing jps to show ghost processes that did not actually exist.

---

6. Duplicate Daemon Processes Running Simultaneously

After multiple failed stop/start attempts, some nodes ended up with two NodeManager or DataNode processes running at the same time, causing resource conflicts and unpredictable behavior.

Solution: Fixed by killing all instances of the daemon by PID directly using 'kill -9', clearing all stale PID files, then doing a clean restart to ensure only one instance was running per service.

---

7. Permission Denied on Hadoop Directories

All Hadoop service startups failed with "Permission Denied" because files were copied as root, leaving root as the owner of all Hadoop directories.

Solution: Ran `chown -R hadoop:hadoop /usr/local/hadoop` on all nodes to give the hadoop user full ownership before starting any services.